

tessera

One Wire Discipline for Identity, Integrity, and Confidentiality
in the Post-Quantum Era

Why TesseraID, TesseraHash, and TesseraSeal each exist, and how they compose

Bahram Samani

tessera monorepo maintainer

Project Whitepaper, Version 1.1

Released 2026-05-21; revised 2026-06-15 (TESS-00023/00024)

Build date: June 17, 2026 **Build time:** 15:13 UTC

Source: `latex/main.tex` **Doc version:** v1.1 (non-normative)

Project artifacts: TesseraID (v1.1, variants `0x01`/`0x02`, 41 B); TesseraHash (v1.0, variant `0x03`, 73 B); TesseraSeal (v1.3, variant `0x04`, suites `0x01`/`0x02`/`0x06`).

Each project has its own normative whitepaper (`tessera-id/latex/TesseraID_Whitepaper.pdf`, `tessera-hash/latex/TesseraHash_Whitepaper.pdf`, `tessera-seal/latex/TesseraSeal_Whitepaper.pdf`). This umbrella document is *non-normative* and does not duplicate their normative content; it covers why the three formats exist, how they compose, and the

shared namespace, discipline, and tooling across them. When it and a project whitepaper disagree, the project whitepaper wins.

Contents

TL;DR **3**

1 Why tessera **4**
1.1 Why three formats, not one . . . 4

2 The Three Projects **5**
2.1 TesseraID: post-quantum-era identifier . . . 5
2.2 TesseraHash: SHAKE-256 content hash . . . 6
2.3 TesseraSeal: ML-KEM-1024 encryption envelope . . . 6
2.4 Common infrastructure . . . 6

3 The Shared Variant-Byte Namespace **7**
3.1 Allocation as of v1.x . . . 7
3.2 The cross-rejection contract . . . 7
3.3 Why one namespace and not three . . . 8
3.4 The 9-byte shared header . . . 8
3.5 Encoding overlap . . . 9

4 Canonical Cross-Project Patterns **9**
4.1 An end-to-end walkthrough . . . 9
4.2 Pattern 1: TesseraID-as-AD (record-level confidentiality binding) . . . 10
4.3 Pattern 2: TesseraHash-as-AD (content-level confidentiality binding) . . . 10
4.4 Pattern 3: TesseraID + TesseraHash (identifier + content commitment) . . . 11
4.5 Pattern 4: TesseraHash of a TesseraSeal envelope (content-addressed ciphertext) . . . 11
4.6 Pattern 5: TesseraID-tagged TesseraHash (provenance-tagged content commitment) . . . 11
4.7 Anti-patterns . . . 12

5 Polyglot Discipline **12**
5.1 The authority order . . . 12
5.2 The eight languages . . . 13
5.3 The cross-language KAT harness . . . 13
5.4 Branch protection . . . 13
5.5 What a spec change looks like . . . 14

6 Conformance and CI **14**
6.1 Repository layout . . . 15
6.2 The hybrid suite 0x06 skip discipline . . . 15
6.3 Latest cross-language state (2026-06-17) . . . 16
6.4 How CI gates the work . . . 16

7 Cross-Project Threat-Model Overview **17**
7.1 What is and isn't "post-quantum" in tessera . . . 17
7.2 Cross-cutting invariants . . . 19
7.3 Side-channel posture . . . 19

7.4 What tessera as a whole does not provide 20
7.5 Pre-external-audit posture 20

8 Roadmap and Scope Discipline **20**
8.1 Per-project release state (as of 2026-05-21) 20
8.2 Open backlog (v1.4 / v2) 20
8.3 Scope discipline 21
8.4 What *would* trigger a v2 of the whole toolkit 22

A Quick Reference Card **23**

B Variant-Byte Allocation Table **23**

C TesseraSeal Suite-Byte Allocation **23**

D Repository Layout **24**

E Author and Document Provenance **24**

TL;DR

tessera is a specification-first, polyglot monorepo housing three post-quantum-era data formats:

- TesseraID (variant bytes `0x01`/`0x02`, 41 B): a sortable, post-quantum-era **identifier**. Pairs a 48-bit ms timestamp with 256 bits of CSPRNG-supplied randomness; the smallest random field that retains 128-bit second-preimage security against Grover's algorithm (Grover, 1996). Two variants: Standard (wall-clock) and HLC (Hybrid Logical Clock) (Kulkarni et al., 2014).
- TesseraHash (variant byte `0x03`, 73 B): a SHAKE-256-based **content identifier**. Pairs the same 9-byte provenance header with a 64-byte (512-bit) SHAKE-256 (fip, 2015) tail. 2^{256} classical collision, $\sim 2^{170}$ BHT (Brassard et al., 1998) quantum collision. Not a hash function; a wire format built on one.
- TesseraSeal (variant byte `0x04`, 16-byte header + variable body): an **encryption envelope**. v1 mandatory suite is ML-KEM-1024 (fip, 2024a) + AES-256-GCM + 32-byte UtC commitment tag + HKDF-SHA3-256. v1.2 added the opt-in hybrid suite `0x06` (X25519 + ML-KEM-1024) for callers who want a classical safety net.

The three projects share one variant-byte namespace, one 9-byte header layout (`TS(6) || VARIANT(1) || CTR(2)`), one cross-language conformance harness at `tools/conformance/run.sh`, one Keccak/SHAKE-256 implementation under `common/`, and one author. Per-project normative content lives in the per-project whitepapers; this document is the cross-project overview.

The integration story

When a record needs a stable identifier, use TesseraID. When a record's content needs an integrity commitment, use TesseraHash. When a record's content needs confidentiality, wrap it in TesseraSeal with the TesseraID (or the TesseraHash) as AEAD associated data. The three formats compose without coupling: a TesseraSeal envelope can carry a TesseraID or a TesseraHash as AD; a TesseraHash can be the AD that binds a TesseraSeal envelope to specific content; a TesseraID can identify a record whose serialized form a TesseraHash commits to and a TesseraSeal envelope encrypts. The three projects exist as siblings, not as a layered cake.

Scope of this umbrella document

This whitepaper does *not* restate the normative content of any per-project whitepaper. It documents only the relationships, shared infrastructure, and canonical cross-project patterns. For TesseraID's 14-section spec, TesseraHash's 10-section spec, or TesseraSeal's 14-section spec, refer to the project-specific PDFs.

Conventions. The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 (Bradner, 1997; Leiba, 2017) when, and only when, they appear in all capitals, as shown here.

1 Why tessera

Two forces are reshaping how large systems handle data, and they pull in the same direction.

Scale-out with no coordinator. Distributed systems — and machine-learning training clusters in particular, now pushing past one million accelerators — mint identifiers on every node with no central allocator in the hot path. Every shard, sample, checkpoint, row, event, and lineage record needs a unique, ideally sortable handle generated *locally*. A worker-ID scheme (Snowflake's 10-bit worker field tops out at 1,024 workers) cannot even enumerate such a fleet; the alternative is a random-field identifier whose field is wide enough that independent generators never collide.

The birthday bound makes this quantitative: for b random bits, the probability that n identifiers contain at least one collision is $P \approx n^2/2^{b+1}$. For a fleet that mints on the order of 10^{18} identifiers over its lifetime:

Format	**Random bits**	$P(\geq 1$ **collision) at** $\sim 10^{18}$ **IDs**
ULID	80	≈ 1 (effectively certain)
UUIDv4	122	$\sim 9\%$
TesseraID	256	$\sim 10^{-41}$ (never)

TesseraID keeps a 48-bit millisecond prefix so identifiers remain time-sortable — index locality on append-heavy checkpoint and sample stores, exactly what UUIDv4 destroys — and spends a full 256 bits on the random tail, putting collisions permanently out of reach regardless of fleet size. UUIDv7 and ULID already brought sortability to random identifiers; tessera's difference is the field width plus a Hybrid-Logical-Clock variant for causal ordering across nodes.

Data that outlives its cryptography. Training corpora, checkpoints, ledgers, and provenance are retained for years — across the expected arrival of cryptographically-relevant quantum computers. tessera sizes every field for that horizon: TesseraID's 256-bit tail retains $\sim$128-bit second-preimage strength against Grover search (an attacker cannot feasibly target or forge a specific record's identifier); TesseraHash carries a 512-bit SHAKE-256 (FIPS 202) content tail for deduplication and integrity across exabyte datasets; and TesseraSeal encrypts artifacts under ML-KEM-1024 (FIPS 203) with a key-committing AEAD, so *harvest-now, decrypt-later* fails against anything sealed today.

The three formats share one variant-byte namespace and one wire discipline, so a single artifact can be *identified* (TesseraID), *content-addressed* (TesseraHash), and *sealed* (TesseraSeal) with primitives that interlock — for example, binding a TesseraID as the AEAD associated data pins an envelope to the record it belongs to.

1.1 Why three formats, not one

A record moving through a large system has three needs that look similar but are not, and no single field serves all three well:

- **Identity** — a name that is unique, time-sortable, and mintable on any node with no coordinator, assigned once and stable for the record's lifetime. This is TesseraID.
- **Integrity** — a fingerprint that changes the instant the bytes change, that any party can recompute with no key and no coordinator, to deduplicate or to verify that received content is the content that was committed. This is TesseraHash.

- **Confidentiality** — a sealed container only the intended recipient can open, built to survive the quantum transition, that cannot be silently re-pointed at a different record. This is TesseraSeal.

They are deliberately *separate* formats because their security arguments and lifetimes diverge: an identifier is public and permanent; a content hash is keyless and rolls on every edit; an envelope is keyed, confidential, and bound to a recipient. Folding them into one field would force the weakest of those assumptions onto all three — a public identifier cannot also be a secret key, and a keyless hash cannot also be a confidentiality boundary. Yet they are *unified* where it pays off: a shared 9-byte header layout, one variant-byte namespace (section 3) so any decoder rejects the other two formats' bytes, and one polyglot conformance harness (section 6). Adopting all three is one mental model and one dependency surface — not three disjoint libraries, pulled from three places, that happen to share a name. The rest of this document is that unifying view: section 2 introduces each format, section 4 shows how they compose end to end, and sections 3, 5 and 6 cover the shared namespace, discipline, and tooling that keep them one system.

2 The Three Projects

Each project under tessera is independent on the wire: a consumer can adopt one without adopting any other, and a downstream system that cares about only identifiers (TesseraID) need not pull in SHAKE-256 (TesseraHash) or ML-KEM-1024 (TesseraSeal). The projects share discipline (Section 5), tooling (Section 6), and a few load-bearing infrastructure pieces (Section 3), but their wire formats commit to nothing about each other beyond non-collision of variant bytes.

2.1 TesseraID: post-quantum-era identifier

Project root: `tessera-id/`. **Whitepaper:** `tessera-id/latex/TesseraID_Whitepaper.pdf`.

TesseraID is a 41-byte, lexicographically-sortable identifier. Layout: `TS(6) || VARIANT(1) || COUNTER(2) || RANDOM(32)`, big-endian. The 32-byte random tail is the security claim: 2^{128} second-preimage even against an attacker running Grover's algorithm (Grover, 1996) — the same security target NIST uses for ML-KEM-1024 (fip, 2024a) (which TesseraSeal uses; see below). The 16-bit per-millisecond counter and 48-bit timestamp make the format sortable and traceable without coordination.

TesseraID v1 ships two variants:

- `0x01` (Standard): wall-clock-driven. Not internally locked; one generator per thread, or caller-locks.
- `0x02` (HLC): Hybrid Logical Clock (Kulkarni et al., 2014). Internally locked. 60-second drift bound: a peer HLC *strictly* more than 60 000 ms ahead of local physical time is a hard error, never clamped. The bound is normative; an implementation MAY expose a stricter-only local override (smaller than 60 000 ms) and MUST reject any attempt to configure a larger one.

Canonical text encodings: 55-character Base64URL (Josefsson, 2006) (no padding) and 66-character Crockford Base32 (Crockford) (case-insensitive on decode, `I/L` → `1` and `O` → `0` confusables accepted). Both round-trip byte-identical with the binary. Crockford ASCII sort preserves generation-

time order.

2.2 TesseraHash: SHAKE-256 content hash

Project root: `tessera-hash/`. **Whitepaper:** `tessera-hash/latex/TesseraHash_Whitepaper.pdf`.

TesseraHash is a 73-byte content identifier. Layout: `TS(6) || VARIANT(1) || COUNTER(2) || TAIL(64)`, big-endian. The 64-byte tail is `SHAKE-256(content, 64 bytes)` (fip, 2015); the 9-byte header is layout-identical to TesseraID's header. The security argument is the SHAKE-256 collision-resistance assumption: 2^{256} classical collision, $\sim 2^{170}$ Brassard–Høyer–Tapp (Brassard et al., 1998) quantum collision, 2^{256} Grover second-preimage.

TesseraHash v1 ships one variant: `0x03`. Canonical text encodings: 98-character Base64URL and 117-character Crockford Base32. TesseraHash is keyless; it provides no authentication, no confidentiality, and no producer non-repudiation (its threat model is closely tied to "the SHAKE-256 sponge is what NIST says it is"). Two callers hashing the same content produce identifiers that share the 64-byte tail and differ only in the 9-byte header.

2.3 TesseraSeal: ML-KEM-1024 encryption envelope

Project root: `tessera-seal/`. **Whitepaper:** `tessera-seal/latex/TesseraSeal_Whitepaper.pdf`.

TesseraSeal is a 16-byte-header variable-body encryption envelope. Header layout: `TS(6) || VARIANT(1) || SUITE(1) || COUNTER(2) || KEM_CT_LEN(2) || AD_LEN(4)`; body: `KEM_CT || AD || NONCE(12) || AEAD_CT || AEAD_TAG(16) || COMMIT(32)`. The first 9 bytes are layout-compatible with TesseraID's and TesseraHash's headers; bytes 9–15 are TesseraSeal-specific.

TesseraSeal v1.x ships three suites:

- `0x01` (mandatory, pure PQ): ML-KEM-1024 + AES-256-GCM + 32-byte UtC commitment tag (Bellare and Hoang, 2022) + HKDF-SHA3-256.
- `0x02` (optional, pure PQ): ML-KEM-1024 + ChaCha20-Poly1305 + UtC + HKDF-SHA3-256. For software-only callers without AES hardware.
- `0x06` (optional, hybrid; v1.2, PQCK-00021): X25519 (Langley et al., 2016) + ML-KEM-1024 + AES-256-GCM + UtC + HKDF-SHA3-256. Opt-in defense-in-depth; uses an X-Wing–style (Connolly et al., 2025) SHAKE-256 combiner over both shared secrets.

The suites `0x01`/`0x02` carry an unambiguous pure-PQ confidentiality promise: residual risk (a structural break of Module-LWE) is accepted in exchange. Suite `0x06` is the opt-in alternative that adds a classical safety net via X25519. The wire byte at header offset 7 records which promise the envelope makes; a single envelope never makes both.

2.4 Common infrastructure

Project root: `common/`. Not a normative wire format; a shared library directory whose contents are used by more than one project.

- `common/c/keccak.{h,c}` (and `common/java/com/tessera/common/Keccak.java`): SHAKE-256/Keccak. Used by TesseraHash (tail computation) and TesseraSeal (HKDF salt derivation;

hybrid suite combiner). Python and Go use stdlib (`hashlib.shake_256`, `golang.org/x/crypto/sha3`); Rust and TypeScript follow each project's pinned per-language sources (audited crates, or runtime-provided OpenSSL-backed primitives — see the per-project pin tables). The `common/` copy is the source of truth for C, C++, and Java.

- `common/c/tessera_header.h`: shared offset constants for the 9-byte header layout (`TS`, `VARIANT`, `COUNTER`). Included by TesseraSeal's C port today; the TesseraID and TesseraHash C ports define the same offsets locally (deliberate duplication, per the `common/` policy below). The first-nine-byte layout itself is identical across all three projects, and the conformance harness keeps it that way.
- `common/c/random.{h,c}`: OS CSPRNG wrapper. Used by TesseraID (random tail) and TesseraSeal (ML-KEM `Encaps` randomness, X25519 ephemeral keygen).
- `common/c/hkdf_sha3.{h,c}`: HMAC-SHA3-256 and HKDF-SHA3-256. TesseraSeal-only as of v1.3, vendored alongside the Keccak source.

The discipline: code lives under `common/` only if more than one project depends on it today. The Keccak source moved to `common/` when TesseraSeal became its second consumer (TesseraHash alone could vendor it inside the project). The HKDF-SHA3 wrapper has not moved out of `common/`-TesseraSeal-territory because no other project uses it.

3 The Shared Variant-Byte Namespace

The single most important piece of shared infrastructure in tessera is not a library — it is a one-byte namespace. The variant byte at offset 6 of every wire format names one of the three projects (or a permanent-reserved sentinel). Cross-project decoders reject every byte they do not own.

3.1 Allocation as of v1.x

Byte	Owner	Meaning	Status
`0x00`	—	Permanently reserved (never assign)	MUST reject
`0x01`	TesseraID	Standard generator	assigned v1.0
`0x02`	TesseraID	HLC generator	assigned v1.0
`0x03`	TesseraHash	SHAKE-256 / 64-byte tail	assigned v1.0
`0x04`	TesseraSeal	Encryption envelope (suite byte at offset 7)	assigned v1.0
`0x05`–`0xFE`	—	Reserved-future	MUST reject
`0xFF`	—	Permanently reserved (never assign)	MUST reject

The reservation of `0x00` and `0xFF` is defensive: zero-filled or all-ones buffers cannot pass validation as any project's wire form. The reserved-future range `0x05`–`0xFE` is available for additional projects under tessera; each new allocation requires a normative whitepaper update and conformance-vector additions to every existing project (to verify cross-rejection).

3.2 The cross-rejection contract

Every project's decoder MUST reject every variant byte it does not own:

- TesseraID decoders accept `0x01` and `0x02` only. Bytes `0x03` (TesseraHash), `0x04` (TesseraSeal),

and every reserved-future byte produce `UnknownVariant` / `TESSERA_ID_E_UNKNOWN_VARIANT`.

- TesseraHash decoders accept `0x03` only. Bytes `0x01`, `0x02`, `0x04`, and reserved-future bytes produce `TESSERA_HASH_E_UNKNOWN_VAR`.
- TesseraSeal decoders accept `0x04` only. Bytes `0x01`, `0x02`, `0x03`, and reserved-future bytes produce `UnknownVariant` / `TESSERA_SEAL_E_UNKNOWN_VARIANT`.

The contract applies at every decode surface, not just the binary one: the Base64URL and Crockford text decoders MUST validate the variant byte at decode time, exactly as the binary decoders do. As of 2026-06-10 all eight language ports of both text-encoded projects (TesseraID, TesseraHash) enforce this, and string-form unknown-variant records in the `invalid.json` KAT files pin it.

The contract is gated by `invalid.json` KAT files in every project's `conformance/vectors/` directory. A decoder that accepts a foreign variant byte fails the cross-language harness; the bug is in the decoder, not in the harness.

3.3 Why one namespace and not three

A naive design would let each project define its own variant byte in its own one-byte field. We chose a single shared namespace for three reasons:

1. **Confusion attacks.** If a decoder sees the first nine bytes of a wire form and the four bytes "timestamp + variant + counter" happen to look valid by every project's local rules, the decoder must decide which project to dispatch to. With a single namespace, the variant byte is the dispatcher; there is no ambiguity.
2. **Substitution attacks.** An attacker who knows a system accepts TesseraIDs as primary keys and TesseraHashes as content keys could substitute one for the other if the variant bytes were locally-scoped. With cross-rejection enforced by the namespace, the substitution fails at the decoder.
3. **Future allocation discipline.** A single, externally-visible byte allocation table makes it impossible to retroactively claim "oh, our project always meant `0x01` to be *this*" — the table is normative across projects, version-controlled, and tested.

3.4 The 9-byte shared header

The first 9 bytes of every project's wire format are layout-compatible: `TS(6) || VARIANT(1) || COUNTER(2)`, big-endian.

- `TS(6)`: 48-bit Unix millisecond timestamp. Every project uses the same constant `MIN_TIMESTAMP_MS = 1_700_000_000_000` as a sanity-check lower bound.
- `VARIANT(1)`: the cross-project namespace byte.
- `COUNTER(2)`: 16-bit big-endian monotonic counter within a millisecond.

The C header at `common/c/tessera_header.h` records those offsets; today it is included by TesseraSeal's C port, while the TesseraID and TesseraHash C ports define the same offsets locally (deliberate duplication, per the `common/` policy of lifting code only once sharing is real — the cross-language harness keeps the layouts identical). Code that just wants to read the timestamp

from any tessera wire form parses the first 6 bytes without knowing or caring which project produced it. Code that wants to dispatch on project reads byte 6.

The 9-byte boundary is where the projects diverge: TesseraID writes a 32-byte random tail, TesseraHash writes a 64-byte SHAKE-256 tail, and TesseraSeal writes a 7-byte body-header continuation (suite + KEM_CT_LEN + AD_LEN) followed by a variable body.

3.5 Encoding overlap

TesseraID and TesseraHash both encode to Base64URL (RFC 4648 §5) and Crockford Base32. The encoder code paths are identical across the two projects in every language (with different alphabet sizes per project). TesseraSeal ships binary-only on the wire: envelope sizes vary, the variable-length text-encoder is more involved than the fixed-length ones, and most callers wrap envelopes with standard Base64URL at the transport layer if they need text.

TesseraID and TesseraHash both guard the trailing padding bits on decode, but the pad widths differ per project because the wire sizes differ. TesseraID (41 bytes = 328 bits): the 55-character Base64URL form ends with a character carrying 4 bits of value plus 2 zero pad bits, and the 66-character Crockford form ends with 3 bits of value plus 2 zero pad bits — in both encodings the final character's low 2 bits MUST be zero. TesseraHash (73 bytes = 584 bits): the 98-character Base64URL form ends with 2 bits of value plus 4 zero pad bits (the final character's low 4 bits MUST be zero, so it is always one of `A`/`Q`/`g`/`w`), and the 117-character Crockford form ends with 4 bits of value plus 1 zero pad bit (the final character's low bit MUST be zero). These guards prevent multiple distinct text strings from decoding to the same binary, which would violate canonical round-trip.

4 Canonical Cross-Project Patterns

The three projects are designed to compose. This section documents the canonical patterns and the reasoning behind each.

4.1 An end-to-end walkthrough

Before the individual patterns, here is the whole chain at once — one training shard moving through a cluster, touching all three formats, with no central coordinator anywhere in the path:

1. **Identity (TesseraID).** The producing node mints a TesseraID for the shard: 41 bytes, generated locally, unique without a coordinator, and time-sortable so the shard lands in order in an append-mostly store.
2. **Integrity (TesseraHash).** It computes a TesseraHash over the shard's bytes. Two nodes that independently hold the same shard derive the same content tail, so duplicates across the corpus collapse to one, and any consumer can verify that the bytes it received are the bytes that were committed — no key, no manifest, no coordinator.
3. **Confidentiality (TesseraSeal).** It seals the shard for the consumer's ML-KEM-1024 public key, binding the TesseraID as the AEAD associated data (Pattern 1 below). The envelope is now post-quantum-confidential *and* cryptographically pinned to that record: an attacker who

re-points it at a different TesseraID gets a generic `DecryptionError`, nothing more.

4. **Content-addressed storage.** The cluster stores the envelope at a key derived from the TesseraHash of the *ciphertext* (Pattern 4 below), so identical sealed objects deduplicate at the storage layer.

5. **Consumption.** A consumer node fetches the envelope, opens it with its private key, and re-derives the TesseraHash of the recovered plaintext to confirm it received exactly the committed content. Identity, integrity, and confidentiality were each provided by exactly one format, and the three composed without a single round-trip to a coordinator.

That is the unifying value: one record, three orthogonal guarantees, one shared wire discipline. The patterns below formalize each link in this chain and name the threat each one closes.

4.2 Pattern 1: TesseraID-as-AD (record-level confidentiality binding)

Use when: a record has a stable TesseraID identifier and the record's content needs confidentiality and integrity tied to that identifier.

```
record_id  = tessera_id.new_hlc()                                  # 41 bytes, variant 0x02
envelope   = tessera_seal.encrypt(plaintext, recipient_pub,
                                  associated_data=record_id)  # bind TesseraID as AEAD AD
# ... after delivery ...
plaintext  = tessera_seal.decrypt(envelope, recipient_priv,
                                  expected_ad=record_id)      # MUST match
```

What it defends: an attacker who reroutes the envelope under a different TesseraID (e.g., to make a leaked envelope appear to belong to a different record) gets AEAD-AD mismatch on decrypt, which produces TesseraSeal's generic `DecryptionError`. The envelope is cryptographically bound to the identifier it was issued for.

Cost: zero additional cryptography. The AEAD AD path is free in every AEAD construction. The TesseraID is 41 bytes; the AEAD AD field has a 64 KiB cap (TesseraSeal `AD_LEN` $\leq$ 65,536).

Test coverage: `tessera-seal/conformance/vectors/ad_binding.json` exercises positive (matching AD decrypts) and negative (any AD bit-flip $\rightarrow$ `DECRYPT_FAILED`) cases.

This is the most-used cross-project pattern, named the "canonical pattern" in TesseraSeal's normative whitepaper.

4.3 Pattern 2: TesseraHash-as-AD (content-level confidentiality binding)

Use when: the encrypted payload must be bound to specific content bytes, not just "some content this recipient can decrypt."

```
content_hash = tessera_hash.hash_only(plaintext)                   # 64 bytes
envelope     = tessera_seal.encrypt(plaintext, recipient_pub,
                                    associated_data=content_hash)
# ... after delivery ...
plaintext    = tessera_seal.decrypt(envelope, recipient_priv,
                                    expected_ad=content_hash)
```

What it defends: an attacker who substitutes the envelope with one carrying different content (but the same recipient key and structurally-valid wire form) fails decryption because the AEAD AD they would supply does not match the embedded content. The pattern composes with UtC commitment (TesseraSeal's 32-byte commitment tag, (Bellare and Hoang, 2022)): UtC closes multi-recipient key confusion; TesseraHash-as-AD closes content substitution. The two bindings are independent.

Variant: instead of passing just the 64-byte SHAKE-256 tail, pass the full 73-byte TesseraHash as AD. This carries provenance (the TesseraHash's timestamp and counter) alongside the content commitment. The choice depends on whether the AD-binding consumer needs the provenance metadata.

4.4 Pattern 3: TesseraID + TesseraHash (identifier + content commitment)

Use when: a record-keeping system tracks both record identity (which mutates rarely) and content (which mutates often).

```
record_id    = tessera_id.new_hlc()           # immutable for the record's
    lifetime
content_hash = tessera_hash.hash_only(...)    # rolls on every content update
# Persist {record_id, content_hash, body} as the canonical record view.
# Subscribers verify both: record_id pins identity, content_hash pins content.
```

What it defends: content-substitution attacks at the storage layer. An attacker who modifies a record's body without rolling the TesseraHash produces inconsistent state that storage-level integrity checks detect. Independent of any cryptographic envelope.

4.5 Pattern 4: TesseraHash of a TesseraSeal envelope (content-addressed ciphertext)

Use when: an object-storage system indexes by ciphertext rather than plaintext.

```
envelope     = tessera_seal.encrypt(plaintext, recipient_pub)
storage_key  = tessera_hash.hash_only(envelope)              # 64-byte
    SHAKE-256
# Store at key=storage_key; lookup by
    key=tessera_hash.hash_only(known_envelope_bytes).
```

What it defends: accidental duplication of identical ciphertexts under different storage keys. TesseraSeal envelopes are deterministic in their inputs (plaintext, recipient key, AD, sender randomness): two envelopes with the same input randomness are byte-identical, so TesseraHash-keying coalesces them. With different sender randomness, two envelopes for the same plaintext differ, so deduplication does not collapse them — this is the right behavior for a system that wants per-encryption auditability.

4.6 Pattern 5: TesseraID-tagged TesseraHash (provenance-tagged content commitment)

Use when: content has multiple producers and the audit log needs to record "producer X committed to content Y at time T."

```
record_id = tessera_id.new_hlc()              # who is committing
content_h = tessera_hash.hash_only(content)   # what they are committing to
audit_entry = sign(record_id || content_h, producer_signing_key)
# Persist audit_entry. Note: TesseraSeal is not involved if the content is
    public;
# wrap with TesseraSeal only when confidentiality is also required.
```

Note: tessera does not (yet) ship a signature primitive. The signing step is performed with an external scheme: ML-DSA (fip, 2024b) for FIPS-aligned PQ signing, Ed25519 (Josefsson and Liusvaara, 2017) for classical, or a hybrid via PASETO (Paragon Initiative Enterprises). A future `pqsign`-style sibling project is outside v1.x scope.

4.7 Anti-patterns

- **Using TesseraID as the AEAD key.** TesseraID is a public identifier, not a key. Anyone who has seen the TesseraID can forge an envelope under it. Use a proper key from a plugin (TesseraSeal's `tessera-seal-plugin-file` or an HSM-backed plugin).
- **Using TesseraHash as a password.** TesseraHash's tail is content-deterministic and keyless. An attacker who can guess the content recomputes the tail. Use Argon2id (Biryukov et al., 2021) or scrypt for password derivation; TesseraHash's threat model explicitly excludes secrecy of low-entropy content.
- **Truncating to fewer bytes than the wire format prescribes.** TesseraID is exactly 41 bytes; TesseraHash is exactly 73 bytes; TesseraSeal's header is exactly 16 bytes. "Optimizing" by dropping fields breaks the wire contract and the cross-language harness.
- **Mixing variant bytes within an artifact.** A 41-byte buffer whose variant byte is `0x03` is not a TesseraID; it is a malformed input that the TesseraID decoder rejects (and the TesseraHash decoder also rejects, because the length is wrong). Don't try to "upgrade" a TesseraID into a TesseraHash by editing one byte.

5 Polyglot Discipline

Every tessera project ships byte-identical reference implementations in eight languages, all consumed by the same cross-language KAT harness. This section documents the discipline that makes that work.

5.1 The authority order

When any two sources disagree, conflicts are resolved in the following order:

1. The project's whitepaper LaTeX (this document for cross-cutting concerns; the per-project document for project-specific normative content).
2. The project's Python reference: `tessera-id/python/tessera_id.py`, `tessera-hash/python/tessera_hash` `tessera-seal/python/tessera_seal.py`.
3. The project's canonical C ABI header: `tessera-id/c/tessera_id.h`, `tessera-hash/c/tessera_hash.h`, `tessera-seal/c/tessera_seal.h`.

4. Every other language port.

A cross-language disagreement is never resolved by changing the spec to match an outlier. The outlier is fixed. "Python is right" is the short version of the rule, with whitepaper-overrides-Python and C-overrides-other-languages as the tie-breakers.

5.2 The eight languages

Every project ships reference implementations in:

Language	Role	Why
Python	Readable canonical reference	The spec is the same shape as the Python; the Python is the test
C	Canonical ABI	Offsets, error codes, function signatures pinned in `tessera_seal`
C++	C-library wrapper	RAII surface over C; no normative content of its own.
Go	Stdlib-heavy port	Tests that the spec is implementable using only stdlib + audited
Rust	Strongest constant-time toolchain	`#![forbid(unsafe_code)]`; tests the safety story.
TypeScript	Browser + Node	Tests that the spec is implementable in JavaScript runtimes.
Java	JVM	Tests that the spec works against JDK stdlib (SunJCE) where ap
CPython extension	Optimized Python	Mirrors the Python API one-to-one; provides a C-fast option with

For TesseraSeal specifically (post-quantum, more moving parts than the other two projects), the alpha language set during v1.x development was Python + C + Go + Rust — the minimum that exercises a readable reference, a canonical ABI, a stdlib-heavy ecosystem, and the strongest constant-time toolchain. C++/TypeScript/Java/CPython ext joined in v1.1.

5.3 The cross-language KAT harness

`tools/conformance/run.sh` is the single entry point. It discovers per-language runners under `tools/conformance/python/`, `tools/conformance/c/`, etc., builds each one if needed, and runs each against every KAT file in `tessera-id/conformance/vectors/`, `tessera-hash/conformance/vectors/`, and `tessera-seal/conformance/vectors/`.

```
# Full run: all eight languages times all three projects
bash tools/conformance/run.sh

# Subset of languages
bash tools/conformance/run.sh python rust go

# Skip languages whose toolchain isn't available locally
TESSERA_CONFORMANCE_SKIP="cpython java" bash tools/conformance/run.sh

# TesseraSeal v1.x alpha quick loop
TESSERA_CONFORMANCE_SKIP="cpython java cpp typescript" bash
    tools/conformance/run.sh
```

5.4 Branch protection

`.github/workflows/conformance.yml` runs the harness on every PR and every push to `main`; branch protection on `main` requires its `cross-language` job, so a red harness blocks merge. It is the only automatically triggered workflow in the repository. The per-language workflows (`python.yml`,

`c.yml`, ...) cover per-OS and per-toolchain variants and are manual-dispatch only; the cross-language harness is the only automatic gate, and the only one that catches polyglot drift.

TesseraSeal has an additional per-project workflow (`tessera-seal.yml`, added in TSEAL-00023; manual dispatch) that runs the TesseraSeal-only matrix across Ubuntu and macOS for the alpha languages plus the LaTeX whitepaper build.

5.5 What a spec change looks like

A normative change to any project follows this order:

1. Update the project's LaTeX whitepaper (and, if cross-cutting, this umbrella document).
2. Update the Python reference.
3. Update the C reference (header first if offsets/constants change, then `.c`).
4. Port to every other language in lockstep. Each must produce byte-identical output for the same input as Python.
5. Regenerate conformance vectors from the Python reference; commit verbatim.
6. Run `bash tools/conformance/run.sh`; confirm green.
7. Write a release-notes entry under `<project>/docs/release-notes/YYYY-MM-DD-<slug>.md`; append to `<project>/docs/release-notes/CHANGELOG.md`.
8. For security-relevant changes, tag the PR `sec(<TICKET>)` and add a "Security considerations" section to the release-notes entry.

Optimization PRs that only touch one language are fine and don't require porting, as long as observable behavior is identical and KAT vectors still pass byte-identical.

6 Conformance and CI

The cross-language conformance harness is the single quality signal that catches polyglot drift. This section documents how it works at the umbrella level; per-project KAT vector inventories live in the per-project whitepapers.

6.1 Repository layout

Path	Contents
`tessera-id/conformance/vectors/`	TesseraID KAT files (decode, encode/decode round-trip, invalid, HLC overflow, min timestamp)
`tessera-hash/conformance/vectors/`	TesseraHash KAT files (decode, hash_only, encode/decode B64URL, encode/decode Crockford, verify, invalid)
`tessera-seal/conformance/vectors/`	TesseraSeal KAT files (decrypt, ad_binding, commitment, invalid, peek_header, min_timestamp, hybrid_*, kyberslash_regression)
`tools/conformance/run.sh`	Single cross-language entry point
`tools/conformance/<lang>/`	Per-language runner sources and binaries
`.github/workflows/conformance.yml`	Automatic gate: runs on every PR and push to `main`; its `cross-language` job is the required branch-protection check
`.github/workflows/tessera-seal.yml`	TesseraSeal-only matrix CI (TSEAL-00023); manual dispatch

6.2 The hybrid suite `0x06` skip discipline

TesseraSeal's hybrid suite `0x06` (PQCK-00021, v1.2) is a polyglot KAT contract carried by the four languages with their own TesseraSeal conformance runners — Python, C (TSEAL-00030, 2026-06-13), Go, and Rust — which decrypt the `hybrid_*.json` vectors byte-identically. The harness filters those vector files per language via the `hybrid_supported()` function in `tools/conformance/run.sh`:

```
hybrid_supported() {
    case "$1" in
        python) return 0 ;;   # PQCK-00021, v1.2
        rust)   return 0 ;;   # PQCK-00028, v1.3
        go)     return 0 ;;   # PQCK-00029, v1.3
        c)      return 0 ;;   # TSEAL-00030, 2026-06-13
        *)      return 1 ;;   # C++ / TS / Java / CPython ext: no seal runner
    esac
}
```

The four languages without a TesseraSeal conformance runner (C++, TypeScript, Java, CPython ext) skip the `hybrid_*.json` files in the harness regardless of library support. This runner axis is distinct from *library* support: as of 2026-06-13, C++ (TSEAL-00031), TypeScript (TSEAL-00032), and the CPython extension (TSEAL-00034) implement suite `0x06` at the library level — each verified by an in-suite test that decodes the frozen `hybrid_*.json` vectors byte-identically — and only Java (TSEAL-00033, backlog) still rejects `0x06` as `UNKNOWN_SUITE`. When porting hybrid into the *harness* for a new language: (i) implement hybrid in that language, (ii) update the language's conformance runner to accept 96-byte hybrid seeds (the standard runners hard-coded `len(seed) != 64`), (iii) flip the language's `hybrid_supported()` entry, (iv) verify the language's PASS count rises by 9 (one per record across the three hybrid KAT files).

6.3 Latest cross-language state (2026-06-17)

Language	PASS	FAIL	SKIP	Notes
Python	298	0	6	TesseraID + TesseraHash + TesseraSeal v1.x including hybrid `0x06` + kyberslash.
C	298	0	6	TesseraSeal hybrid via OpenSSL 3.5+ `EVP_PKEY_X25519` (TSEAL-00030).
C++	229	0	6	TesseraID + TesseraHash only (no TesseraSeal runner yet).
Go	298	0	6	TesseraSeal hybrid via stdlib `crypto/ecdh` (PQCK-00029).
Rust	298	0	6	TesseraSeal hybrid via `x25519-dalek` 2 (PQCK-00028).
TypeScript	229	0	6	TesseraID + TesseraHash only (no TesseraSeal runner yet).
Java	229	0	6	TesseraID + TesseraHash only (no TesseraSeal runner yet).
CPython ext	224	0	11	TesseraID + TesseraHash; HLC clock-override records SKIP explicitly.

The four alpha languages each gained one invalid record since 2026-06-10 from the TSEAL-00049 `KEM_CT_LEN` pin (288 standard records), one `kyberslash_regression.json` record, and C rose from 287 to 298 by consuming the 9 `hybrid_*.json` records once its hybrid port landed (TSEAL-00030): 288 standard + 9 hybrid + 1 kyberslash = 298.

C++, TypeScript, Java, and the CPython extension have no `tessera_seal_runner` binary and *skip* the TesseraSeal vector directory entirely — there is *no* Python-runner fallback for TesseraSeal (`tessera_seal_runner_cmd` in `tools/conformance/run.sh` simply has no entry for those four languages). Their TesseraSeal conformance coverage begins when their v1.4 TesseraSeal ports land; their per-language suites cover TesseraID and TesseraHash directly today. SKIP counts are explicit, never silent: `doc` pseudo-records and records a language cannot execute emit `SKIP` lines that the harness reconciles against each file's record count.

6.4 How CI gates the work

`conformance.yml` is the only automatically triggered workflow in the repository: it runs on every pull request and every push to `main` (plus manual dispatch), and branch protection on `main` requires its `cross-language` job. Within that job the blocking steps are the four-language alpha run (Python + C + Go + Rust — the four ports with their own TesseraSeal runners) and a TesseraSeal decrypt-path timing-histogram gate (`tools/scripts/timing-histogram.py -profile ci`); the full eight-language step is informational (`continue-on-error`) until the remaining TesseraSeal ports land. The ten other workflows (per-language matrices, `tessera-seal.yml`, `latex.yml`) are manual-dispatch only, for cost control. The harness is non-flaky by design: KAT vectors are deterministic, the per-language runners do not call out to the network, and the toolchains are pinned. A red harness means one of two things:

- A language port produced a different byte sequence for the same input. The port is wrong; fix the port.
- The KAT vector itself changed. Shipped vectors are frozen append-only: regeneration MAY add records or files but MUST NOT mutate existing record bytes, and a mutated record fails the freeze check. A Python-reference behavior change is a spec change, requires a release-notes entry, and all other language ports must be updated to match before the PR merges.

The harness is what makes the spec-first methodology load-bearing. Without it, "Python is right" is just a slogan; with it, a language port that disagrees with Python is detectable in seconds.

7 Cross-Project Threat-Model Overview

Each project has its own normative threat model in its own whitepaper. This section summarizes the differences and the shared ground.

7.1 What is and isn't "post-quantum" in tessera

The "post-quantum" in tessera means different things in different projects, and the differences matter.

Project	**What “PQ” refers to**	**Binding security argument**
TesseraID	*Post-quantum-era hygiene.*	“The OS CSPRNG produces unguessable bits, and 256 of them is enough.” The 256-bit random tail retains 128-bit second-preimage against Grover’s algorithm (Grover, 1996). TesseraID is not a cryptographic primitive; it is an identifier whose random field is sized for the era in which quantum

The first two are not making cryptographic claims — they are public artifacts. An attacker sees every byte of every TesseraID and TesseraHash; the security of TesseraID reduces to "the random tail is unguessable" and the security of TesseraHash reduces to "the SHAKE-256 hash is collision-resistant." Both projects defer producer authentication, integrity of the wire form, and confidentiality of their contents to an outer envelope.

TesseraSeal is the only project that makes a cryptographic confidentiality claim. The claim is structured carefully: suites `0x01`/`0x02` make a *pure post-quantum* promise (no classical fallback; if ML-KEM-1024 is structurally broken, every envelope under these suites loses confidentiality). Suite `0x06` makes a *hybrid* promise (an attacker must break both ML-KEM-1024 *and* X25519 to recover the session key). The wire byte at offset 7 records which promise applies; a single envelope never makes both.

7.2 Cross-cutting invariants

These apply to all three projects:

- **OS CSPRNG only.** Where randomness is needed (TesseraID's 32-byte tail; TesseraSeal's ML-KEM `Encaps` randomness and X25519 ephemeral keygen for hybrid), it comes from the OS CSPRNG: `getrandom(2)` / `getentropy(3)` / `BCryptGenRandom` / `os.urandom` / `SecureRandom` / `crypto.getRandomValues` / `/dev/urandom`. No userspace PRNG, no fallback. TesseraHash has no random field and is exempt.
- **Variant-byte cross-rejection.** Each project's decoders reject every variant byte they do not own (Section 3). The contract is gated by KAT vectors.
- **No unsafe Rust, no cgo, no new FFI.** Implementations are pure within their language. CPython extensions are direct C against the CPython API, not FFI bridges.
- **Supply-chain hygiene.** No new third-party crypto dependencies. SHAKE-256/Keccak comes from stdlib (Python, Go) or vendored `common/` (C, C++, Java). TesseraSeal's ML-KEM-1024 and X25519 sources are pinned per language; see TesseraSeal's whitepaper §5.

7.3 Side-channel posture

- TesseraID and TesseraHash are *public artifacts* with no secret-dependent control flow. Side-channel attacks on the generator recover, at most, content the attacker may already have submitted (TesseraHash) or the rate of identifier generation (TesseraID).
- TesseraSeal mitigates *software-timing* side channels by pinning verified constant-time ML-KEM-1024 and X25519 implementations per language and by collapsing every authentication-class decrypt failure into a single `DecryptionError`. (The TesseraSeal whitepaper's §9.2 synthetic-key timing-pad mandate is an open spec-vs-implementation alignment item tracked in the v1.2 security-audit notes.) TesseraSeal explicitly does *not* mitigate power, electromagnetic, frequency-scaling (Hertzbleed), or prefetcher (GoFetch) side channels; the normative side-channel disclaimer is at `tessera-seal/docs/side-channel-disclaimer.md`.

7.4 What tessera as a whole does not provide

- **No signature primitive.** Producer authentication is out of v1.x scope. Use ML-DSA (fip, 2024b), Ed25519 (Josefsson and Liusvaara, 2017), or PASETO (Paragon Initiative Enterprises). A future `pqsign`-style sibling project is outside v1.x scope.
- **No transport layer.** tessera ships wire formats, not protocols. Use TLS 1.3, QUIC, or Noise underneath.
- **No FIPS 140-3 module validation.** The algorithms used in TesseraSeal are FIPS-approved (FIPS 203 ML-KEM, FIPS 197 AES, FIPS 202 SHA-3 family); the implementation is not a validated module. Callers who need module validation should source ML-KEM from a validated module (e.g., a future BoringCrypto release) and integrate at the API boundary.
- **No key rotation primitive.** Each TesseraSeal envelope is self-contained; rotation is a higher-level concern that the plugin protocol and the outer application own.

7.5 Pre-external-audit posture

TesseraSeal v1.x ships with an internal pre-external-audit threat-model walk-through at `tessera-seal/docs/securi`
It enumerates known attacks vs mitigations, per-language posture, and six open items for an external auditor; of these, the CI timing histogram (automated in TSEAL-00048) and the hybrid C port (TSEAL-00030, 2026-06-13) have since shipped, leaving four (FIPS 140-3 path, synthetic-key timing-pad invariant testing, plugin protocol fuzz harness, memory-zeroization audit).

TesseraID and TesseraHash are simpler in shape and do not have an equivalent internal audit document; their threat models live entirely in their respective whitepapers. A future external audit would cover all three projects, with the TesseraSeal portion being the largest by surface area.

8 Roadmap and Scope Discipline

This section documents what is in scope for v1.x, what is scheduled for v1.4/v2, and what is explicitly out of scope.

8.1 Per-project release state (as of 2026-05-21)

Project	Version	Date	Major milestones
TesseraID	v1.1	2026-05-21	Eight-language byte-identical implementations; HLC variant; cross-project variant-byte rejection contract pinned.
TesseraHash	v1.0	2026-05-21	Eight-language byte-identical implementations; SHAKE-256 tail; cross-rejection of cross-project bytes.
TesseraSeal	v1.3 (+hybrid)	2026-06-13	Eight-language alpha for suites `0x01`/`0x02`; hybrid `0x06` as a 4-way conformance contract (Python, Rust, Go, C) plus library-level hybrid in C++, TypeScript, CPython ext; plugin protocol + `.tseal-key-v1.1` keyfile; CLI binary.

8.2 Open backlog (v1.4 / v2)

TesseraSeal hybrid `0x06` port status (as of 2026-06-13):

1. TSEAL-00030: Hybrid suite `0x06` port to C (OpenSSL 3.5+ `EVP_PKEY_X25519`). **Shipped** — promotes hybrid to a 4-way byte-identical conformance contract (Python, Rust, Go, C) and closes the largest single gap.
2. TSEAL-00031: Hybrid `0x06` C++ wrapper (routes to the C entry points). **Shipped** — library-level, verified by in-suite `hybrid_*.json` KAT byte-identity (no TesseraSeal conformance runner yet).
3. TSEAL-00032: Hybrid `0x06` in TypeScript (`@noble/curves` X25519, pinned 2.0.1). **Shipped** — library-level, verified by in-suite KAT byte-identity.
4. TSEAL-00034: Hybrid `0x06` in the CPython extension (wraps the C reference). **Shipped** — library-level, verified by in-suite KAT byte-identity.
5. TSEAL-00033: Hybrid `0x06` in Java (SunJCE `XDH` keypair). **Open** — the sole remaining hybrid gap, pending a JDK 24+ build/verify environment.

Other TesseraSeal backlog:

1. Add TesseraSeal conformance runners for C++, TypeScript, Java, and the CPython extension so their (already library-level) suite coverage — including hybrid `0x06` where implemented — is harness-enforced rather than in-suite-test-enforced.
2. Rust AES-NI/ARMv8 intrinsics for 1 MiB AEAD throughput. Currently regressed because `aes-gcm 0.10` does not enable ARMv8 intrinsics on stable Rust without a nightly cfg.
3. Suite `0x07` (HQC contingency) is reserved on the registry (TSEAL-00048) but *not* allocated; v1.x decoders MUST reject it. Allocation is gated on NIST HQC FIPS finalization or a Module-LWE confidence loss.

TesseraID v1.x: no open backlog. The format is stable; future work is operational (benchmarks, additional reference implementations, documentation polish).

TesseraHash v1.x: CPython extension is the largest single gap; mirrors the Python API.

Cross-project: sibling project `pqsign` (variant byte `0x05`? or further into the reserved-future range) is an explicit *out-of-v1.x* candidate. The minimal scope would be ML-DSA (fip, 2024b) signatures of a payload-with-context, using a TesseraID or TesseraHash as the canonical payload. Not committed; documented as a possibility so the reserved-future range is interpretable.

8.3 Scope discipline

- **No new variant bytes without a whitepaper update.** `0x00` and `0xFF` are permanently reserved; `0x05`–`0xFE` are reserved-future. Any new allocation requires (a) a normative whitepaper update in the owning project, (b) conformance-vector additions to every existing project verifying cross-rejection, and (c) sign-off as a security-relevant change.
- **No third-party crypto dependencies without justification.** Each project's whitepaper pins per-language sources for its primitives. New dependencies require explicit justification in the PR description.

- **No unsafe Rust, no cgo, no FFI shortcuts.** Implementations stay pure within their language. The CPython extensions are C-against-CPython-API, not FFI bridges.
- **No format-version mutations of an existing variant.** A variant byte *is* a version. A change to the layout, the security argument, or the algorithm choices under an existing byte is a spec break. New algorithms get new bytes — and the same rule applies one level down in TesseraSeal's *suite*-byte namespace at header offset 7: the hybrid construction took the new suite byte `0x06` inside variant `0x04` rather than mutating suite `0x01`.

8.4 What *would* trigger a v2 of the whole toolkit

- **A structural break of Module-LWE.** Would force a successor envelope format: a hypothetical TesseraSeal v2 takes a new *variant* byte allocated from the reserved-future range `0x05`–`0xFE`. (Note the namespace distinction: `0x06` is a *suite* byte inside variant `0x04`'s suite namespace at header offset 7, not a variant byte — the two one-byte namespaces are disjoint, so the variant-byte allocation table in Section 3 is unaffected by suite-byte assignments.) TesseraID and TesseraHash are unaffected by ML-KEM breaks because they make no PQ-cryptography claim.
- **A serious break of SHAKE-256.** Would force TesseraHash to a new variant byte with a different hash function (KMAC256? a stream cipher in PRF mode? a different sponge?). TesseraSeal's KDF schedule would also need revision because HKDF-SHA3-256 depends on SHAKE-256. TesseraID is unaffected.
- **A serious break of AES-256-GCM.** Would force TesseraSeal to a new suite under a new byte using a different AEAD. TesseraID and TesseraHash are unaffected.
- **A change to the variant-byte allocation discipline.** Would force every project to a new wire-format version, indicated by a new variant byte. This is the worst-case scenario and is exactly what the immutable-variant-byte rule is designed to prevent.

None of these is anticipated in the v1.x lifetime. The roadmap is conservative on purpose: tessera is intended to outlive any individual deployment, which means most of its versions look like "a new entry was added to the variant table; everything else is unchanged."

A Quick Reference Card

Property	**TesseraID**	**TesseraHash**	**TesseraSeal**
Variant byte	`0x01` / `0x02`	`0x03`	`0x04` (+ suite byte at offset 7)
Wire size	41 B fixed	73 B fixed	16-byte header + variable body
Tail / body	32 B CSPRNG random	64 B SHAKE-256	`KEM_CT || AD || nonce || CT || tag || COMMIT`
Primitive	OS CSPRNG	SHAKE-256 (FIPS 202)	ML-KEM-1024 (+ X25519 for `0x06`) + AES-256-GCM + UtC
Security claim	128-bit Grover second-preimage	2^{256} classical / $\sim 2^{170}$ BHT collision	ML-KEM-1024 IND-CCA2 + AEAD + UtC commitment
Public artifact?	Yes (no secrets)	Yes (no secrets)	No (carries confidential plaintext)
Text encodings	55-char Base64URL, 66-char Crockford	98-char Base64URL, 117-char Crockford	Binary only (v1)
Round-trip determinism	N/A (random)	Yes (content-deterministic)	Suites `0x01`/`0x02`: yes, given a fixed RNG seed (how KAT decrypt vectors are generated). Hybrid `0x06`: encryption draws fresh X25519 ephemerals from the OS CSPRNG; its KAT files pin captured envelopes, frozen append-only

B Variant-Byte Allocation Table

Byte	**Owner**	**Project / variant**	**First spec version**	**Status**
`0x00`	—	Permanently reserved (defensive)	—	MUST reject
`0x01`	TesseraID	Standard generator	v1.0	assigned
`0x02`	TesseraID	HLC generator	v1.0	assigned
`0x03`	TesseraHash	SHAKE-256 / 64-byte tail	v1.0	assigned
`0x04`	TesseraSeal	Encryption envelope (suite byte at offset 7)	v1.0	assigned
`0x05`–`0xFE`	—	Reserved-future	—	MUST reject
`0xFF`	—	Permanently reserved (defensive)	—	MUST reject

C TesseraSeal Suite-Byte Allocation

(Header offset 7 within a TesseraSeal envelope; only valid when `VARIANT == 0x04`.)

Suite	Algorithms	Status
0x01	ML-KEM-1024 + AES-256-GCM + UtC commit + HKDF-SHA3-256	v1.0 mar
0x02	ML-KEM-1024 + ChaCha20-Poly1305 + UtC commit + HKDF-SHA3-256	v1.0 opti
0x06	X25519 + ML-KEM-1024 + AES-256-GCM + UtC commit + HKDF-SHA3-256	v1.2 opti
0x00, 0x03–0x05, 0x07–0xFF	Reserved-future	MUST r

D Repository Layout

```
tessera/
|-- CLAUDE.md                   # umbrella discipline file
|-- README.md                   # repo entry point
|-- latex/                      # this umbrella whitepaper
|   `-- main.tex, sec*.tex, references.bib, Makefile
|
|-- tessera-id/                       # Project 1: identifier
|   |-- CLAUDE.md, README.md
|   |-- latex/                  # TesseraID whitepaper sources
|   |-- python/, c/, cpp/, go/, rust/, typescript/, java/, cpython/
|   |-- conformance/vectors/    # KAT files
|   |-- benchmarks/             # per-language bench data
|   `-- docs/                   # release notes, docs site sources
|
|-- tessera-hash/                     # Project 2: content hash
|   |-- (mirror TesseraID layout; latex/ added in TesseraHash v1.0)
|   `-- ...
|
|-- tessera-seal/                   # Project 3: encryption envelope
|   |-- (mirror TesseraID layout)
|   |-- plugin/                 # plugin protocol + reference plugin
|   `-- docs/                   # incl. side-channel-disclaimer.md,
    security-audit-v1.2.md
|
|-- common/                     # shared library code
|   |-- c/                      # keccak, tessera_header, random, hkdf_sha3
|   `-- java/                   # Keccak.java
|
`-- tools/
    `-- conformance/            # cross-language KAT harness + per-language
        runners
```

E Author and Document Provenance

Author: Bahram Samani, tessera monorepo maintainer.

This umbrella whitepaper is built from LaTeX sources under `latex/` (`main.tex` + `secNN_*.tex`); the build chain is `pdflatex` → `bibtex` → `pdflatex` × 2 per the Makefile. The artifact at `latex/Tessera_Whitepaper.pdf` is regenerated and committed alongside any normative change to the umbrella.

Per-project documents. This umbrella does not restate normative content from the per-project

whitepapers. Refer to:

- TesseraID: `tessera-id/latex/TesseraID_Whitepaper.pdf`. 46-page document covering wire format, security analysis, distributed-systems model, comparisons with UUIDv4/v7/ULID/KSUID/Snowflake, eight-language reference implementations.
- TesseraHash: `tessera-hash/latex/TesseraHash_Whitepaper.pdf`. 21-page document covering wire format, SHAKE-256 binding, encodings, security argument, conformance, comparison with adjacent hash formats.
- TesseraSeal: `tessera-seal/latex/TesseraSeal_Whitepaper.pdf`. 26-page document covering wire format, ML-KEM-1024 binding, hybrid suite `0x06`, AEAD + UtC commitment, KDF, AD-binding patterns, failure modes, side channels, plugin protocol, conformance, comparisons.
- This umbrella document is the repo-root published artifact (`Tessera_Whitepaper.pdf`); it is *non-normative* relative to the three project whitepapers above (when they disagree, the project whitepaper wins).

Authority order (when sources disagree): the per-project whitepaper for project-specific content; this umbrella whitepaper for cross-project content; then the per-project Python reference; then the per-project canonical C ABI; then every other language port. A cross-language disagreement is never resolved by changing the spec to match an outlier.

Conformance gate: `tools/conformance/run.sh`, run automatically by `.github/workflows/conformance.yml` on every PR and push to `main`; branch protection on `main` requires that workflow's `cross-language` job (the 4-language alpha step and the TesseraSeal timing-histogram step block). Latest cross-language run (2026-06-17): the four alpha languages report Python 298 / C 298 / Go 298 / Rust 298 PASS, 0 FAIL (1192 PASS total) across all three projects — includes the append-only `kyberslash_regression.json` record; the full eight-language run is also green, with C++/TypeScript/Java at 229 PASS each and the CPython extension at 224 PASS + 11 SKIP (those four have no TesseraSeal runner yet and skip the TesseraSeal vector directory; all skips are explicit).

Acknowledgments. Bellare and Hoang for the UtC commitment construction. The OpenSSL, RustCrypto, Go core team, `kyber-py`, SunJCE, and `@noble` maintainers for the cryptographic primitive sources TesseraSeal pins per language. The Cryspen team for `mlkem-native`, evaluated during C-backend selection. The X-Wing draft authors (Connolly et al., 2025) for the combiner design idiom TesseraSeal's hybrid suite borrows. The KyberSlash, PQShield, and GoFetch disclosure teams, whose findings shape TesseraSeal's side-channel disclaimer. The SHAKE-256 / SHA-3 standardization team (Bertoni, Daemen, Peeters, Van Assche) whose primitive is the load-bearing element of every tessera project.

References

SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, National Institute of Standards and Technology, Aug. 2015. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS PUB 203,

National Institute of Standards and Technology, Aug. 2024a. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf.

Module-lattice-based digital signature standard. Technical Report FIPS PUB 204, National Institute of Standards and Technology, Aug. 2024b. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf.

M. Bellare and V. T. Hoang. Efficient schemes for committing authenticated encryption. In *Advances in Cryptology – EUROCRYPT 2022*, 2022. URL https://eprint.iacr.org/2022/1260.

A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson. The memory-hard Argon2 password hash and proof-of-work function. RFC 9106, IETF, 2021. URL https://www.rfc-editor.org/rfc/rfc9106.

S. Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, BCP 14, IETF, 1997. URL https://www.rfc-editor.org/rfc/rfc2119.

G. Brassard, P. Høyer, and A. Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN '98*, 1998. URL https://arxiv.org/abs/quant-ph/9705002.

D. Connolly, P. Schwabe, and B. Westerbaan. X-Wing: General-purpose hybrid post-quantum KEM. draft-connolly-cfrg-xwing-kem, IETF CFRG, 2025. URL https://datatracker.ietf.org/doc/draft-connolly-cfrg-xwing-kem/.

D. Crockford. Base32. URL https://www.crockford.com/base32.html.

L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996. URL https://arxiv.org/abs/quant-ph/9605043.

S. Josefsson. The Base16, Base32, and Base64 data encodings. RFC 4648, IETF, 2006. URL https://www.rfc-editor.org/rfc/rfc4648.

S. Josefsson and I. Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017. URL https://www.rfc-editor.org/rfc/rfc8032.

S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone. Logical physical clocks. In *Principles of Distributed Systems (OPODIS)*, 2014. URL https://cse.buffalo.edu/tech-reports/2014-04.pdf.

A. Langley, M. Hamburg, and S. Turner. Elliptic curves for security. RFC 7748, IETF, 2016. URL https://www.rfc-editor.org/rfc/rfc7748.

B. Leiba. Ambiguity of uppercase vs lowercase in RFC 2119 key words. RFC 8174, BCP 14, IETF, 2017. URL https://www.rfc-editor.org/rfc/rfc8174.

Paragon Initiative Enterprises. PASETO: Platform-agnostic security tokens, v4. URL https://github.com/paseto-standard/paseto-spec.